

UHack-a-Py

Bullet-hell template

0. Introductie

Hier zal jullie worden aangeleerd hoe je de **Bullet-Hell** template aanvult tot een werkend startpunt. Bullet-Hell is een genre van games waarbij er zeer veel projectielen op het scherm staan die de speler dan moet ontwijken of zelf afvuurt op de vijanden. Enkele bekende voorbeelden zijn **Enter the Gungeon**, **Vampire Survivors** en **Brotato**, maar er zijn nog veel meer gelijkaardige games.

De bedoeling van deze template is om ook zelf zo een game te maken. We beginnen simpel bij het bewegen van de speler en bouwen het steeds verder op. Zo zullen we nog zien hoe we de vijanden laten bewegen en kogels kunnen schieten en verplaatsen.



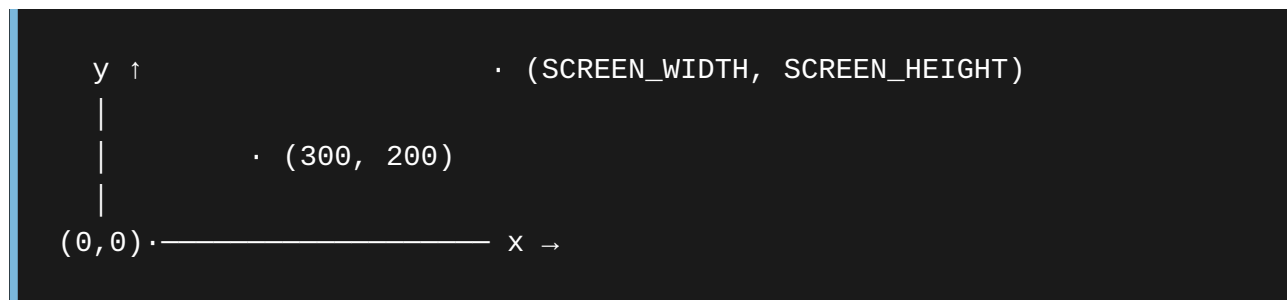
Het eindresultaat zal er uitzien zoals op de foto hierlangs. Dit is echter gewoon **een start waarop jullie verder kunnen bouwen**.

Om dit te gaan maken krijgen jullie een systeem op basis van **PyGame**, een Python library voor het maken van games. Enkele basislijnen van de code zijn ook al voorzien, die hoeven jullie niet meer aan te passen om de template werkende te krijgen.

Om een goed begrip te krijgen van dit systeem en de templates raden we aan om deze cursus goed te volgen. Als er vragen zijn of iets niet duidelijk is, **aarzel niet om hulp te roepen!**

Het coördinatenstelsel

Voordat we beginnen is het belangrijk om te begrijpen hoe posities op het scherm werken. Het punt **(0, 0)** ligt **linksonder** op het scherm. De x-waarde loopt naar rechts en de y-waarde loopt **naar boven**.



Wil je iets helemaal rechtsboven tekenen? Dan gebruik je een x-waarde dicht bij **SCREEN_WIDTH** en een y-waarde dicht bij **SCREEN_HEIGHT**. Dit is een veelgemaakte fout, dus hou het goed in de gaten!

De gegeven code

In de template zijn al een aantal variabelen en functies voor je aangemaakt. Hieronder vind je een overzicht van de belangrijkste zodat je weet wat er al beschikbaar is. **Je hoeft deze niet aan te passen om de opdrachten te voltooien**, maar je bent vrij om waarden te wijzigen als je het spel sneller, groter of anders wilt maken.

Variabele	Betekenis	Standaardwaarde
player_position	Positie van de speler als Vector (x, y)	Midden van het scherm
player_speed	Snelheid waarmee de speler beweegt	5.5
player_size	Breedte en hoogte van de speler in pixels	150
player_direction	Richting waarin de speler beweegt als Vector	Vector(0, 0)
enemies	Lijst van vijandposities (elk een Vector)	Lege lijst

<code>enemy_speed</code>	Snelheid van de vijanden	2
<code>enemy_size</code>	Grootte van een vijand in pixels	150
<code>bullet_positions</code>	Lijst van kogelposities (elk een Vector)	Lege lijst
<code>bullet_directions</code>	Lijst van kogelrichtingen (elk een Vector)	Lege lijst
<code>bullet_speed</code>	Snelheid van de kogels	15
<code>bullet_cooldown</code>	Timer die bijhoudt wanneer je opnieuw mag schieten	0
<code>score</code>	Huidige score van de speler	0

Naast de variabelen zijn er ook functies al voor je voorzien:

Functie	Wat ze doet
<code>spawn_enemies()</code>	Laat vijanden verschijnen aan de randen van het scherm
<code>detect_collisions()</code>	Berekent botsingen tussen speler, kogels en vijanden
<code>remove_bullets_out_of_screen()</code>	Verwijdert kogels die buiten het scherm zijn
<code>show_game_over()</code>	Toont het game-over scherm en herstart het spel

Frames

In een game wordt het scherm **niet één keer getekend**, maar **meerdere keren per seconde**. Elke keer dat het scherm opnieuw wordt getekend noemen we een **frame**.

In deze template draait het spel aan **30 frames per seconde (30 FPS)**. Dat betekent dat het scherm **30 keer per seconde opnieuw wordt opgebouwd**.

Een game werkt door een **oneindige loop** die steeds opnieuw dezelfde stappen uitvoert zolang het spel draait.

In de template nemen we voor iedere frame de volgende stappen aan:

```
Frame:  
  1. Input lezen  
  2. Posities aanpassen  
  3. Alles opnieuw tekenen
```

Bijvoorbeeld:

```
player_position.x += player_speed
```

Omdat deze verandering elke frame gebeurt, lijkt het alsof de speler vloeiend over het scherm beweegt.

1. Speler laten bewegen

Een spel begint meestal met het bewegen van een speler, maar voordat we daar aan kunnen beginnen moeten we eerst de speler kunnen zien.

De speler tonen

In de basislijnen van de code staat het begin van de spellogica waarbij we starten met:

```
while playing():
```

Alles wat daaronder valt zal automatisch elke **frame** worden uitgevoerd. Een frame is wat je op één moment op je scherm ziet. Het systeem draait altijd aan **30 fps (frames per second)**, wat betekent dat alles in de while-loop 30 keer per seconde wordt uitgevoerd. Een simpel voorbeeld: als je elke frame 1 optelt bij de x-positie van de speler, dan beweegt de speler elke seconde 30 pixels naar rechts.

Op het einde van die while-loop zal de **tekenfase** beginnen. In die fase kun je afbeeldingen en figuren tonen op het scherm. Het werkt als **een echt canvas**: het eerste

dat je tekent, ligt achter de rest. Zo zien we in de template dat de achtergrond als eerste wordt getekend en daarna pas de tekst.

Om iets op het scherm te tekenen heb je **3 opties**: **rectangle**, **circle** en **image**. Dat zijn functies die je kan aanroepen, net zoals de print-functie van Python. Het aanroepen gaat zoals hieronder aangegeven.

```
rectangle(x, y, width, height, color)
circle(x, y, radius, color)
image(image_name, x, y, width, height)
```

Voor ze allemaal heb je een **x**, **y**, **width** en **height** of **radius** nodig, die vanzelfsprekend voor de positie en grootte van de figuur dienen. Voor de rectangle en circle heb je ook een **color** nodig, daarvoor zijn er enkele kleuren voorzien. Zo kan je bijvoorbeeld **colors.RED** typen om een rode kleur te geven aan de figuur.

Opdracht 1 — in de tekenfase (# TODO: Opdracht 1): Probeer nu een lichtgroene cirkel te tekenen linksonder op het scherm. Doe dit in de tekenfase van de code. Let er op dat de cirkel volledig in het scherm past! Gebruik onderstaande afbeelding als referentie (het moet niet exact hetzelfde zijn).

Tip: Je kan **colors.** typen in je programmeeromgeving om alle mogelijke kleuren te zien. Vergeet niet: (0, 0) is **linksonder**, dus voor linksonder gebruik je een x en y waarde die hier in de buurt liggen.



In plaats van een lichtgroene cirkel laten we wel graag een afbeelding zien als speler. Daarvoor zullen we de `image`-functie gebruiken. De meeste parameters zijn hetzelfde als bij de `rectangle` en `circle`, maar `image_name` is nieuw.

Daar kunnen we eenvoudig de bestandsnaam van de afbeelding aan geven als een `String`. Het is wel belangrijk dat de afbeelding in de juiste folder zit, in het voorziene systeem is dat de **"images"-folder**.

Daar zijn al enkele afbeeldingen in voorzien, gebruik voor de speler de **"mage.png"**.

Opdracht 2 — in de tekenfase (# TODO: Opdracht 2, 4, 8): Toon de speler met een image op zijn positie met een bepaalde grootte, gebruik hiervoor de juiste variabelen (deze zijn reeds aangemaakt in de template). De cirkel uit opdracht 1 mag hier verwijderd worden.

Tip: Gebruik `player_position.x` en `player_position.y` voor de positie en `player_size` voor de breedte en hoogte.

De speler bewegen

Nu we de speler zien kunnen we beginnen met het te laten bewegen. We zullen dat doen aan de hand van de pijltjestoetsen. Om invoer te vragen van de gebruiker is er weer een functie voorzien in het systeem, namelijk de `input` functie. PyGame voorziet namen voor de toetsen op het toetsenbord, beginnend met `"K_"`. Zo kan je opvragen of er op de linkerpijltjestoets wordt gedrukt met:

```
if input(K_LEFT):
```

Daarna kunnen we eenvoudig 1 aftrekken van de x-waarde en zal de speler bewegen. Op die manier beweegt de speler echter zeer traag, daarom maken we gebruik van grotere waarden die we best instellen in een variabele, zoals `player_speed`.

Opdracht 3 — in de spellogica (# TODO: Opdracht 3): Laat de speler bewegen met behulp van de pijltjestoetsen door de juiste variabelen aan te passen

met `player_speed`. Voor nu mag je de `player_direction` nog achterwege laten.

Zorg er ook voor dat de speler niet buiten het scherm kan bewegen. Je kunt dit doen door de x- en y-positie te beperken tussen 0 en respectievelijk `SCREEN_WIDTH - player_size` en `SCREEN_HEIGHT - player_size`. Controleer met `if-statements` of de positie onder het minimum of boven het maximum komt en zet ze dan terug binnen de grens.

Test checkpoint: Op dit punt zou je een werkende speler moeten hebben die je kunt zien en in alle richtingen kunt bewegen met de pijltjestoetsen. Test dit door het spel te runnen en te controleren of de speler soepel beweegt en binnen het scherm blijft. Laat jullie werk even controleren voordat je verder gaat!

2. Vijanden

Het spel is zeker niet compleet zonder vijanden, dat is dus onze volgende stap. In principe weten jullie al alles om dit te kunnen maken, maar hieronder volgt nog wat uitleg om jullie verder op weg te helpen.

De vijanden tonen

We beginnen weer met het tekenen op het beeld. Om de vijanden te laten zien zullen ze ook in het spel moeten spawnen (= inladen). Dat wordt al voor jullie gedaan door het aanroepen van de `spawn_enemies()` functie. De functie vult de lijst van enemies aan, telkens met hun positie als Vector. Zo kan de lijst er bijvoorbeeld uit zien als:

```
enemies = [Vector(0, 500), Vector(187, 550), Vector(130, 10)] # Een vo
```

In dit voorbeeld zijn er 3 vijanden, elk met hun eigen positie. Elke vijand wordt voorgesteld als een Vector met twee waarden: de x-positie en de y-positie.

Opdracht 4 — in de tekenfase (# TODO: Opdracht 2, 4, 8): Toon alle vijanden die in de `enemies`-lijst zitten. Gebruik hiervoor de `"enemy.png"` afbeelding en de variabelen `enemy_size` voor de grootte.

Tip: Je zal een for-loop moeten gebruiken om elke enemy te doorlopen: `for enemy in enemies:`. Gebruik dan `enemy.x` en `enemy.y` als positie.

Test checkpoint: Op dit punt zou je vijanden moeten zien spawnen en getekend worden op het scherm. Test dit door het spel te runnen en te controleren of vijanden verschijnen op verschillende posities.

De vijanden bewegen naar de speler

Voor de vijanden te laten bewegen doen we weer hetzelfde als bij de speler: we passen de positie aan met de `enemy_speed`. Deze keer kunnen we geen gebruikmaken van de input-functie, aangezien de vijanden vanzelf moeten bewegen. In plaats daarvan moeten we de positie van de vijand vergelijken met de positie van de speler.

Opdracht 5 — in de spellogica (# TODO: Opdracht 5): Laat alle vijanden vanzelf bewegen naar de speler.

Tip: Gebruik een for-loop om elke enemy te doorlopen. Vergelijk voor elke enemy de positie met de speler. Vijanden kunnen zowel links, rechts, boven als onder de speler staan. Als de vijand links van de speler staat (`enemy.x < player_position.x`), dan moet de x-positie verhoogd worden met `enemy_speed`. Analoog voor de andere richtingen.

Als jullie dit werkende hebben gekregen zullen jullie misschien al hebben gezien dat de speler sterft als hij een enemy aanraakt. Dat is ook een belangrijk deel van het vijandensysteem, dat al door de gegeven code wordt geregeld.

Test checkpoint: Op dit punt zou je vijanden moeten zien bewegen naar de speler toe. Probeer te bewegen en te zien of de vijanden je volgen!

3. Kogels

De game is zo bijna compleet, maar een bullet-hell zonder bullets is niet veel fun. Als de laatste stap zullen we dus het kogelsysteem maken. Dit is zeker ook het moeilijkste deel, dus aarzel niet om vragen te stellen!

Kogels schieten

Deze keer is het spawnen van de kogels nog niet voor jullie gemaakt. Voordat we kogels kunnen aanmaken moeten we weten in welke richting de speler loopt. Daarvoor gaan we even terug naar de code uit opdracht 3. Deze keer mogen we de `player_direction` niet meer achterwege laten. Aan de hand van die variabele slaan we de richting op voor latere berekeningen.

Opdracht 6 — in de spellogica (# TODO: Opdracht 6, 7, 9): Sla de richting op in `player_direction` door het x-element op -1 te zetten voor links en 1 voor rechts, analoog voor het y-element. Als er geen invoer is, sla dan een 0 op.

Tip: Je kan `player_direction` printen om te controleren of het werkt:

```
print(player_direction.x, player_direction.y)
```

Nu we de richting kennen kunnen we een nieuwe kogel aanmaken. Een kogel wordt voorgesteld met twee Vectors:

```
bullet_position = Vector(x, y)
bullet_direction = Vector(x_richting, y_richting)
```

Daarna voeg je de positie en richting toe aan beide lijsten:

```
bullet_positions.append(bullet_position)
bullet_directions.append(bullet_direction)
```

Opdracht 7 — in de spellogica (# TODO: Opdracht 6, 7, 9): Voeg telkens een nieuwe kogel toe aan de lijsten zoals hierboven beschreven. Let erop dat een kogel met `bullet_direction.x` én `bullet_direction.y` gelijk aan 0 niet mag bestaan — een kogel zonder richting kan niet bewegen! Controleer dit met: `if player_direction.x != 0 or player_direction.y != 0`.

Kogels doen bewegen

Een kogel doen bewegen is nu niet meer zo moeilijk. Als we de richting van de kogel toevoegen aan de positie (vermenigvuldigd met `bullet_speed`), beweegt de kogel voor altijd in de afvuurrichting.

Opdracht 8 — in de spellogica én de tekenfase

(# TODO: Opdracht 6, 7, 9 en # TODO: Opdracht 2, 4, 8): Laat alle kogels bewegen in de afvuurrichting. Vergeet ook niet de kogels toe te voegen aan de tekenfase zodat je ze kan zien.

Test checkpoint: Op dit punt zou je kogels moeten kunnen schieten en zien bewegen. Test dit door het spel te runnen, te bewegen, en te schieten. Controleer of kogels in de juiste richting bewegen en verdwijnen wanneer ze buiten het scherm zijn.

Cooldowns voor schieten

Bij het testen merken jullie misschien dat de speler heel erg snel schiet. Om dat op te lossen maken we gebruik van een **cooldown**. Telkens wanneer je schiet start je een timer. Die telt elke frame af, en zodra hij terug op 0 staat mag je opnieuw schieten. Hiervoor gebruiken we de `bullet_cooldown`, die standaard op 0 staat.

Door de `bullet_cooldown` op bijvoorbeeld 30 te zetten na het schieten, starten we een timer van 1 seconde (want de game draait op 30 fps). Daarna testen we of de cooldown 0 is voordat we de speler laten schieten, en anders trekken we 1 af van de cooldown.

Opdracht 9 — in de spellogica (# TODO: Opdracht 6, 7, 9): Zorg ervoor dat de cooldown werkt voor het schieten zoals hierboven beschreven.

Tip: Volg deze stappen:

1. Wanneer de speler schiet, zet je `bullet_cooldown` op een waarde (bijv. 30 voor 1 seconde).
2. Elke frame: als de cooldown groter is dan 0, verlaag je hem met 1.
3. Je mag alleen schieten als de cooldown op 0 staat.

Test checkpoint: Test dit door snel de schietknop in te houden. Je zou nu een duidelijke vertraging moeten merken tussen de kogels.

4. Eindresultaat

Als je alle stappen hebt gevolgd heb je nu een volledig functioneel spel, goed gedaan! Laat zeker alles even controleren zodat je niets mist voordat we verder gaan met uitbreidingen!

In de volgende fase zullen jullie het spel mogen uitbreiden met jullie eigen ideeën. Om jullie toch wat op weg te helpen zijn hier nog enkele voorstellen:

- Momenteel schiet de speler in de richting waarin hij loopt. Pas de code aan zodat de speler altijd naar de positie van de muiscursor schiet.
- Laat vijanden soms een item vallen als ze sterven. Als de speler dit oppakt, krijgt hij een bonus, bijvoorbeeld:
 - **Machinegeweer:** Verlaag de `bullet_cooldown` tijdelijk zodat je supersnel kunt schieten.
 - **Shotgun:** Schiet 3 kogels tegelijk in een waaivorm.
 - **Speed boost:** Verhoog tijdelijk de `player_speed`.
- Maak variaties van vijanden, zoals een "Tank" die langzaam is maar meerdere kogels nodig heeft, of een "Sniper" die stilstaat maar kogels op jou afvuurt.

Het belangrijkste blijft dat jullie je eigen ideeën tot leven brengen, dus laat je niet beperken door deze voorstellen!